

Slovak University of Technology in Bratislava

Faculty of Informatics and Information Technologies

NAO ROBOT EXERCISE SYSTEM

Technical Methodology

Team Leaders

Ing. Jakub Perdek, PhD.
doc. Ing. Fedor Lehocki PhD.

Team Members

Bc. Akhmadinurov Amal
Bc. Danilov Denis
Bc. Dmitriev Anton
Bc. Gainetdinov Ruslan
Bc. Kuzin Timofei
Bc. Tikhoblazhenko Alena
Bc. Zimnitski Aliaksei

Bratislava • 2026/2027

Contents

1 Coding conventions **2**

1.1 General rules 2

1.2 Python services 2

1.3 Web application and configuration 3

2 Version control **3**

3 Pull request and code review **4**

3.1 Before requesting review 4

3.2 Reviewer responsibilities 4

3.3 Merge gate 5

4 Verification levels **6**

5 Publishing team documents **6**

6 Maintaining this methodology **7**

References **8**

Project: NAO Robot Exercise System

Scope: Source code, configuration, documentation, and changes to robot behaviour.

This methodology applies to every contributor to the NAO system. The repository contains Python services, a TypeScript web application, Docker configuration, and a legacy Python 2 service that communicates with NAO. The rules below define how changes are written, tracked, reviewed, and merged.

1 Coding conventions

1.1 General rules

1. Make each change as small as the task allows. Keep unrelated refactoring in a separate issue and pull request (PR).
2. Use descriptive English names for files, functions, variables, API fields, and commits. Prefer names that express a unit or purpose, such as `duration_seconds` instead of `duration`.
3. Keep modules focused. Separate robot control, video processing, HTTP routes, and data storage so each can be tested independently.
4. Do not commit passwords, API keys, `.env` files, local paths, generated media, caches, virtual environments, or build artifacts. Add required settings to an example configuration with safe placeholder values.
5. Keep URLs, IP addresses, ports, camera paths, and exercise storage paths configurable. Do not introduce a developer's machine-specific settings into shared code.
6. When changing an API request, response, or stored exercise format, update its consumers and document the change in the same PR. Preserve compatibility where practical; otherwise describe the migration.
7. Write comments for non-obvious constraints and decisions. Prefer clear code over comments that merely repeat what a line does.

1.2 Python services

- Follow PEP 8 style: four-space indentation, `snake_case` for functions and variables, `PascalCase` for classes, and explicit imports.

- Keep new Python 3 code compatible with the version declared by its service. Do not assume that all services use the same Python version.
- Isolate Python 2 and NAOqi-specific code in the NAO adapter/service. New Python 3 modules must not import Python 2-only packages directly.
- Catch specific exceptions at service boundaries. Log enough context to diagnose a failure without logging secrets or unnecessary user video/audio data.
- Add or update automated tests for changed behaviour. Use a fake NAO interface for routine tests; physical movement is tested separately on the robot.

1.3 Web application and configuration

- Use the repository's TypeScript and ESLint configuration. Resolve lint and type-check errors introduced by the change before requesting review.
- Keep UI components focused and use explicit types for data crossing API boundaries.
- Validate Compose and deployment configuration after edits. Do not use mutable image tags such as `latest` for a reproducible release.
- Keep example configuration and run instructions usable on Windows, macOS, and Linux. Document any platform-specific exception.

2 Version control

1. `master` is the integration branch. Create a short-lived branch from the current `master` for each Jira task. Name it `<jira-key>-short-description`, for example `NAO-42-video-trimming`. If the team changes its integration branch, update this document.
2. Link every code change to a Jira issue. The issue states the goal and acceptance criteria before implementation begins.
3. Commit coherent changes with a subject in the form `<type>(<scope>): <summary>`, for example `fix(translator): reject invalid pose data`. Use `feat`, `fix`, `test`, `docs`, `refactor`, `build`, or `chore` as the type. Put the Jira key in the PR title or description.

4. Do not commit directly to `master`. Open a PR for the task, keep it focused, and synchronize it with `master` before merge if conflicts arise.
5. Do not force-push or rewrite shared branch history. On a personal task branch, rebasing is allowed while nobody else depends on it; tell reviewers when a rebase invalidates their previous review.
6. Merge only after the acceptance criteria are met, required checks pass, and review is approved. Prefer squash merge to keep one traceable commit per PR; the PR remains the record of intermediate discussion.
7. Tag versions used for demonstrations or NAO experiments. Record the tag or commit SHA with the experiment results so the software state can be reproduced.

3 Pull request and code review

3.1 Before requesting review

The author shall:

- Review their own diff for accidental files, secrets, unrelated changes, debug code, consistent indentation, descriptive names, and unhandled failure paths.
- Link the Jira issue and explain **what changed**, **why**, **how it was tested**, and any remaining limitation.
- Include test output or clear manual verification steps. State explicitly when a change could not be tested on NAO.
- Add a screenshot or short recording for a visible UI change when that makes review easier.
- Describe any change to robot motion, safety limits, actuator use, camera handling, or service interfaces prominently in the PR.
- Request review from a team member familiar with the affected component. Move the Jira issue to **In review** when the PR is ready.

3.2 Reviewer responsibilities

At least one team member other than the author reviews every PR. The reviewer must understand the change well enough to assess it; pairing during implementation

is welcome, but the final diff still receives an independent review. A change that can move the robot or alter safety-related behaviour also requires review by a team member responsible for robot testing. Reviewers check:

1. Does the implementation satisfy the Jira acceptance criteria without unrelated scope?
2. Are service boundaries and API/data contracts clear and compatible with consumers?
3. Are failure paths handled, logged, and testable without physical hardware?
4. Do tests cover the changed behaviour, and are manual steps sufficient where automation is impossible?
5. Are configuration, dependency, privacy, and robot-safety implications documented?
6. Do representative valid and invalid inputs behave as expected? When practical, reproduce at least one relevant path locally rather than relying only on reading the diff.

Review comments should identify the issue and a suggested direction. The reviewer records findings in the PR, requests changes when necessary, and tells the author when the review is complete. The author responds to every blocking comment with a fix or an explanation; the reviewer checks the revised code again before resolving the comment. Share the outcome with the team through the PR and Jira issue. Do not approve a PR solely because automated checks passed.

3.3 Merge gate

A PR may be merged when it has an independent approval, all blocking comments are resolved, required automated checks pass, and the PR description records the relevant test result. Only then is the Jira issue moved to **Done**. If review finds a blocking problem, return the issue to **In progress**, fix it, and request another review. If the work cannot be completed within the sprint, move the unfinished issue to the next sprint instead of marking it Done.

Code that requires physical NAO testing may merge after offline verification **only if** the PR and Jira issue mark the robot test as pending and the change is not enabled for a live session until that test passes. Any safety-related change must be tested on the robot before being used in a demonstration.

4 Verification levels

Table 1: Verification levels

Level	When	Evidence
Static checks	Every PR	Lint, type checks where available, and configuration validation
Unit tests	Behaviour changes	Automated test result with external services and NAO mocked
Service integration	Changes across services	Repeatable test with sample data and fake NAO responses
Physical NAO test	Movement, balance, temperature, or hardware integration changes	Date, operator, robot state, software commit/tag, scenario, result, and incident notes

Automated CI must never issue movement commands to a physical NAO. Schedule hardware tests in advance because robot access is intermittent. The team member responsible for the robot defines safe movement limits and stop procedures; this methodology does not replace those procedures.

5 Publishing team documents

The `naomi-documents` GitHub repository is the source for PDF files published on the team website. Put each document in a directory under its document type: `zapisnice` for meeting minutes, `retrospektivy` for retrospectives, `progres` for progress reports, `metodiky` for methodologies, `prirucky` for handbooks, or `vystupy` for project deliverables. Each document directory contains `main.tex` and `document.json`. The metadata file supplies the English and Slovak names and may specify an ISO date; otherwise the publication date comes from the last Git change to the document or shared assets.

When editing in Overleaf, download the project's source ZIP, replace the source files in the corresponding GitHub directory, and push through a normal pull request. This manual source transfer works with the repository's multiple document projects. Overleaf's direct GitHub synchronization is a premium, manually triggered feature that links one project to one repository; a separate mirror per project may be configured later. Do not upload a generated PDF to the website by hand.

On each pull request, GitHub Actions compiles every categorized LaTeX project and

produces a review artifact containing the PDFs and `documents.json`. It places these outputs into a temporary website checkout, checks the links, and builds the website. A failed compilation or site build blocks publication. After the reviewed change reaches `main`, the workflow updates the website's PDFs and index with a repository-scoped publication token. The website's own workflow then deploys the new version. Verify the PDF artifact during review and check the published links after the first rollout. A bad publication can be rolled back by reverting the generated website commit.

6 Maintaining this methodology

Changes to these rules are made through a PR and reviewed by the team. Review the document at least once per semester and after a serious integration or robot-test incident. Record the decision in the PR so contributors can see why a rule changed.

References

[1] Python Software Foundation. “Pep 8 – style guide for python code.” (), [Online]. Available: https://peps.python.org/pep-0008/ (visited on 10/22/2026).

[2] Microsoft. “The typescript handbook.” (), [Online]. Available: https://www.typescriptlang.org/docs/handbook/intro.html (visited on 10/22/2026).

[3] S. Chacon and B. Straub. “Pro git: Branching workflows.” (), [Online]. Available: https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows (visited on 10/22/2026).

[4] GitHub. “Pull request reviews.” (), [Online]. Available: https://docs.github.com/en/pull-requests/reference/pull-request-reviews (visited on 10/22/2026).